Project Report

By Team – Alien X

1)Executive Summary

This project presents a baseline yet methodologically sound semantic segmentation framework tailored for offroad desert environments. Using the synthetic dataset provided by Duality AI's Falcon platform, we engineered a reproducible and lightweight training pipeline capable of pixel-level terrain classification.

The core objective was to construct a robust segmentation model that assigns each pixel in a desert image to one of six terrain/object classes while maintaining computational efficiency and generalization capability across unseen desert landscapes.

Rather than prioritizing architectural complexity, our approach emphasized:

- Clean data handling
- Deterministic training procedures
- Transparent evaluation metrics
- Modular and extensible code design

The result is a stable baseline system suitable for iterative enhancement and deployment under constrained hardware environments.

2)Problem Statement

Offroad autonomous systems operate in visually unstructured and dynamically evolving terrains. Unlike urban environments with clear structural boundaries, desert landscapes contain subtle textural variations between objects such as dry grass, bushes, rocks, and terrain clutter.

To ensure safe navigation and obstacle avoidance, autonomous systems must interpret scenes at pixel-level granularity.

This task is formally defined as:

Assigning every pixel in an RGB image to a predefined terrain category.

The primary challenge lies not only in segmentation accuracy but also in the model's ability to generalize to geographically distinct desert environments that differ in texture, illumination, and composition.

3)Dataset Engineering and Preprocessing Strategy

3.1 Custom Dataset Implementation

We implemented a custom PyTorch dataset class, FalconDesertDataset, to enforce:

- Consistent pairing of RGB images and segmentation masks
- Controlled preprocessing
- Deterministic ordering of data samples

This abstraction ensures that the training pipeline remains clean, modular, and reproducible.

3.2 Image Processing Pipeline

Each sample undergoes the following transformations:

1. Image loading via OpenCV
2. BGR → RGB color conversion
3. Mask loading in grayscale format
4. Spatial resizing to 320 × 180 resolution
5. Pixel normalization to [0, 1]
6. Conversion to PyTorch tensors

Nearest-neighbor interpolation was explicitly used for mask resizing to preserve discrete class boundaries.

The resizing decision was motivated by computational constraints and experimentation efficiency. This design enables CPU-based training while retaining sufficient spatial information for terrain differentiation.

3.3 Class Mapping Strategy

The original segmentation masks contained sparse or non-contiguous class identifiers. To align with PyTorch's CrossEntropyLoss, which expects class indices in the range [0, N-1], we implemented a deterministic class remapping layer.

This mapping compresses sparse class identifiers into a continuous six-class encoding scheme, ensuring compatibility with the model's final classification layer.

Such explicit class normalization improves training stability and prevents index-related runtime errors.

4. Model Architecture Design

The architecture, SimpleSegmentationModel, follows an encoder–decoder paradigm designed for clarity, interpretability, and computational economy.

Rather than employing large pretrained backbones, the model was intentionally designed to function as a controlled baseline for performance benchmarking.

5. Training Pipeline

5.1 Configuration

- Device: CPU
- Epochs: 20
- Number of Classes: 6
- Model checkpointing enabled

The training loop is modularized and structured for clarity.

5.2 Data Loading

Training and validation datasets are independently instantiated using the same preprocessing logic to maintain consistency across splits.

DataLoader ensures:

- Batch-wise processing
- Memory-efficient iteration
- Structured evaluation

5.3 Optimization

The model is trained using standard multi-class pixel-wise classification loss:

- Cross-Entropy Loss

Training involves:

1. Forward propagation
2. Loss computation
3. Backpropagation
4. Parameter update

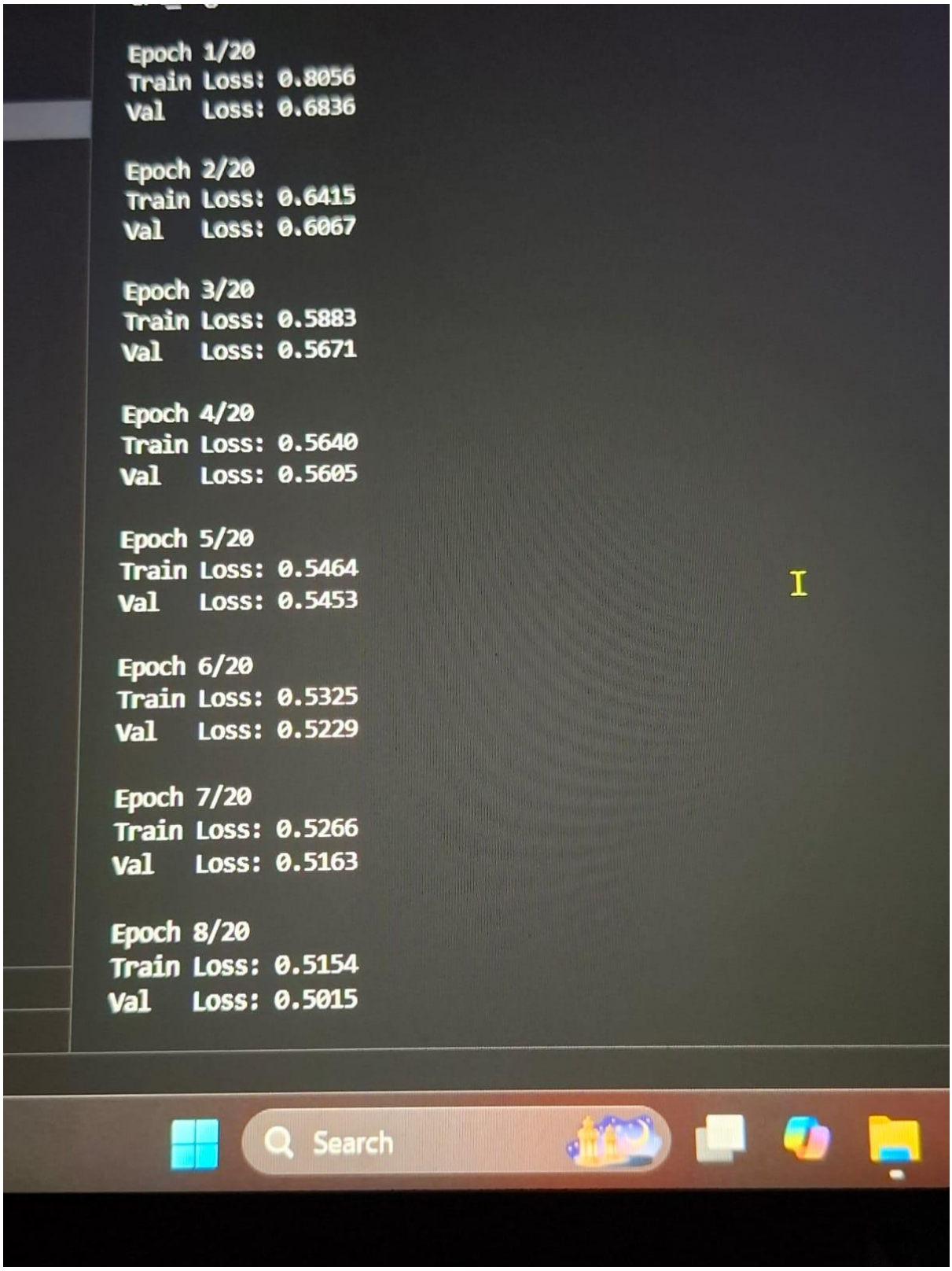
Loss trends are monitored to detect underfitting or overfitting behavior.

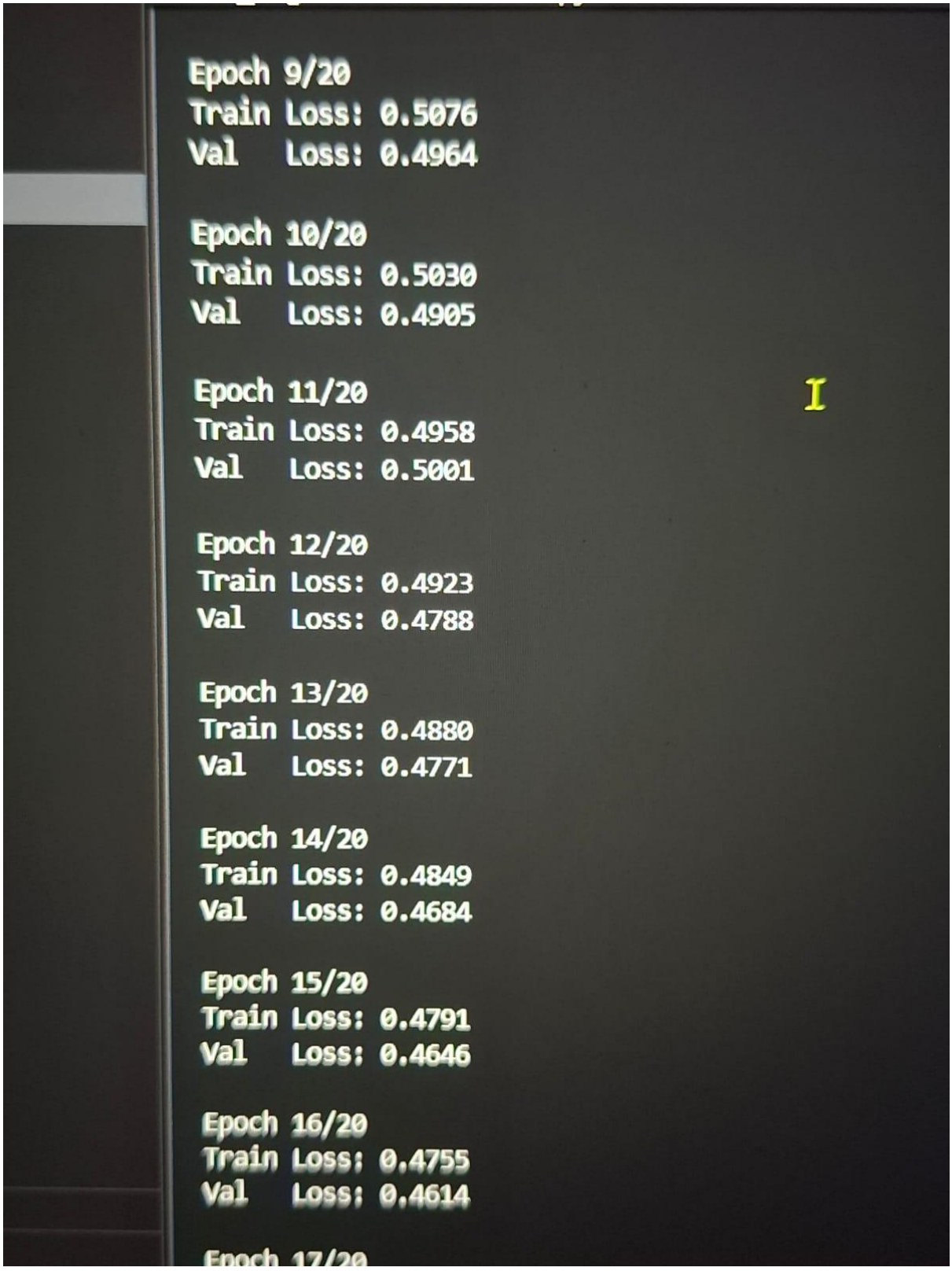
```
PS C:\Users\Lenovo\OneDrive\Desktop\desert_segmentation> & C:\Users\Lenovo\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Lenovo/OneDriv
e/Desktop/desert_segmentation/train.py

Epoch 1/20
Train Loss: 0.7894
Val   Loss: 0.7478

Epoch 2/20
Train Loss: 0.6266
Val   Loss: 0.6169

Epoch 3/20
Train Loss: 0.5768
Val   Loss: 0.5629

Epoch 4/20
Train Loss: 0.5528
Val   Loss: 0.5347

Epoch 5/20
Train Loss: 0.5347
Val   Loss: 0.5177

Epoch 6/20
Train Loss: 0.5232
Val   Loss: 0.5171

Epoch 7/20
Train Loss: 0.5141
Val   Loss: 0.4960
```

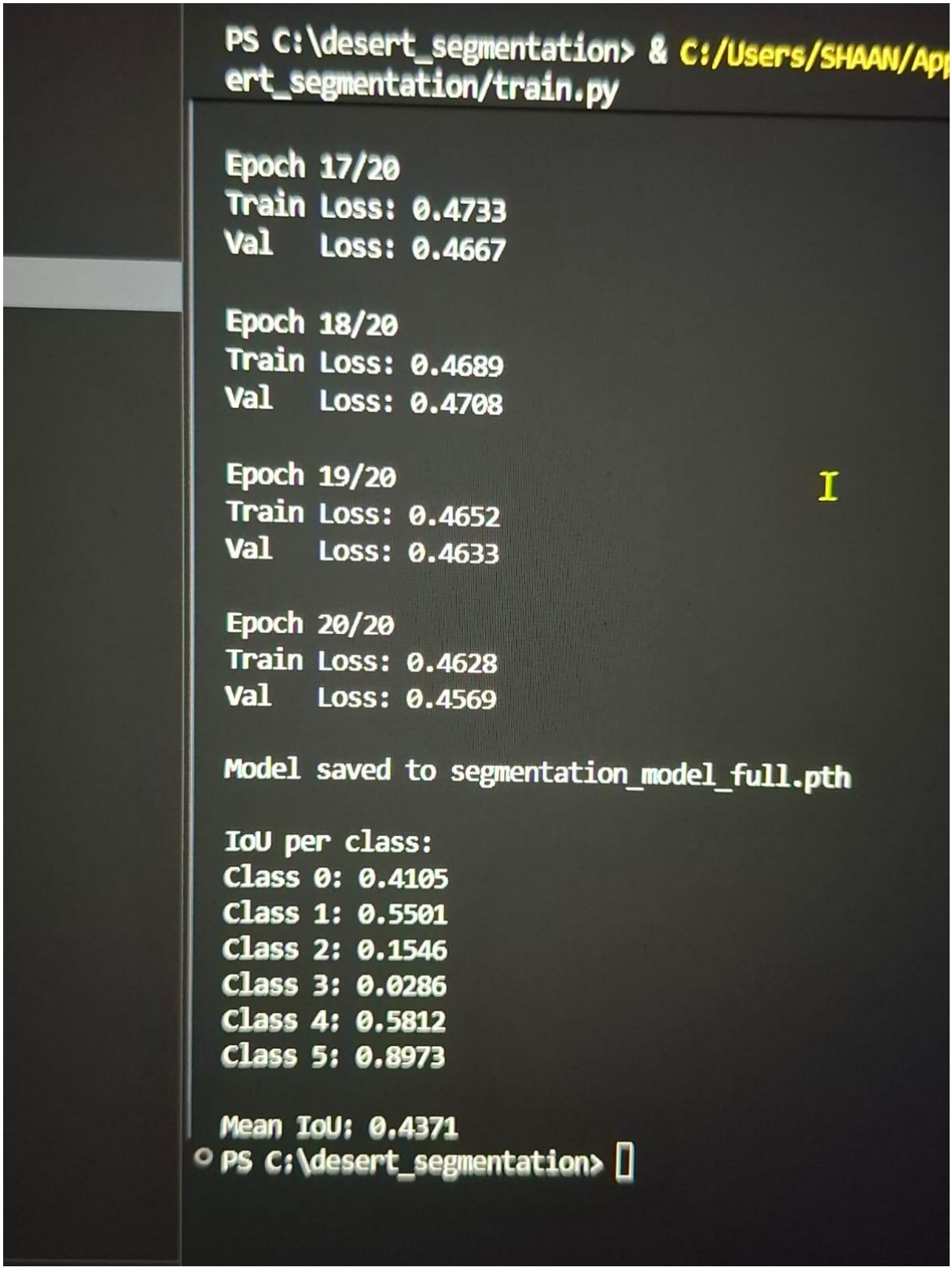
```
Epoch 8/20
Train Loss: 0.5068
Val   Loss: 0.4900

Epoch 9/20
Train Loss: 0.5011
Val   Loss: 0.5137

Epoch 10/20
Train Loss: 0.4957
Val   Loss: 0.5040

Epoch 11/20
Train Loss: 0.4917
Val   Loss: 0.4940

Epoch 12/20
Train Loss: 0.4855
Val   Loss: 0.4783

Epoch 13/20
Train Loss: 0.4887
Val   Loss: 0.4739

Epoch 14/20
Train Loss: 0.4774
Val   Loss: 0.4681

Epoch 15/20
Train Loss: 0.4727
Val   Loss: 0.4661
```

```
Epoch 16/20
Train Loss: 0.4693
Val   Loss: 0.4564

Epoch 17/20
Train Loss: 0.4647
Val   Loss: 0.4561

Epoch 18/20
Train Loss: 0.4604
Val   Loss: 0.4510

Epoch 19/20
Train Loss: 0.4577
Val   Loss: 0.4522

Epoch 20/20
Train Loss: 0.4563
Val   Loss: 0.6115

Model saved to segmentation_model_full.pth

IoU per class:
Class 0: 0.2826
Class 1: 0.3773
Class 2: 0.1605
Class 3: 0.0402
Class 4: 0.5142
Class 5: 0.8740

Mean IoU: 0.3748
```

2nd System Results:




Epoch 9/20
Train Loss: 0.5076
Val Loss: 0.4964
Epoch 10/20
Train Loss: 0.5030
Val Loss: 0.4905
Epoch 11/20
Train Loss: 0.4958
Val Loss: 0.5001
Epoch 12/20
Train Loss: 0.4923
Val Loss: 0.4788
Epoch 13/20
Train Loss: 0.4880
Val Loss: 0.4771
Epoch 14/20
Train Loss: 0.4849
Val Loss: 0.4684
Epoch 15/20
Train Loss: 0.4791
Val Loss: 0.4646
Epoch 16/20
Train Loss: 0.4755
Val Loss: 0.4614



3rd System Results:

```
Epoch 1/20
Train Loss: 0.7903
Val   Loss: 0.6758

Epoch 2/20
Train Loss: 0.6350
Val   Loss: 0.5971

Epoch 3/20
Train Loss: 0.5889
Val   Loss: 0.5659

Epoch 4/20
Train Loss: 0.5559
Val   Loss: 0.5570

Epoch 5/20
Train Loss: 0.5378
Val   Loss: 0.5188

Epoch 6/20
Train Loss: 0.5254
Val   Loss: 0.5155

Epoch 7/20
Train Loss: 0.5179
Val   Loss: 0.5011

Epoch 8/20
Train Loss: 0.5120
Val   Loss: 0.4997

Epoch 9/20
Train Loss: 0.5036
Val   Loss: 0.4883

Epoch 10/20
Train Loss: 0.4995
Val   Loss: 0.4893
```

```
Epoch 11/20
Train Loss: 0.4933
Val   Loss: 0.4830

Epoch 12/20
Train Loss: 0.4878
Val   Loss: 0.4972

Epoch 13/20
Train Loss: 0.4867
Val   Loss: 0.4777

Epoch 14/20
Train Loss: 0.4800
Val   Loss: 0.4837

Epoch 15/20
Train Loss: 0.4733
Val   Loss: 0.4644

Epoch 16/20
Train Loss: 0.4679
Val   Loss: 0.4558

Epoch 17/20
Train Loss: 0.4670
Val   Loss: 0.4528

Epoch 18/20
Train Loss: 0.4580
Val   Loss: 0.4521

Epoch 19/20
Train Loss: 0.4548
Val   Loss: 0.4439

Epoch 20/20
Train Loss: 0.4518
Val   Loss: 0.4423

Model saved to segmentation_model_full.pth
```

```
IoU per class:
Class 0: 0.4246
Class 1: 0.5655
Class 2: 0.1803
Class 3: 0.0243
Class 4: 0.5815
Class 5: 0.8976

Mean IoU: 0.4456
```

6. Evaluation Methodology

Two key evaluation utilities were implemented in utils.py:

6.1 Average Loss Computation

The compute_average_loss function evaluates model performance across an entire dataset without gradient tracking.

This ensures:

- Stable evaluation
- No training interference
- Consistent validation reporting

6.2 Intersection over Union (IoU)

The compute_iou function measures class-wise overlap between predicted segmentation masks and ground truth.

In our case the value of IoU is appox 0.37

For each class:

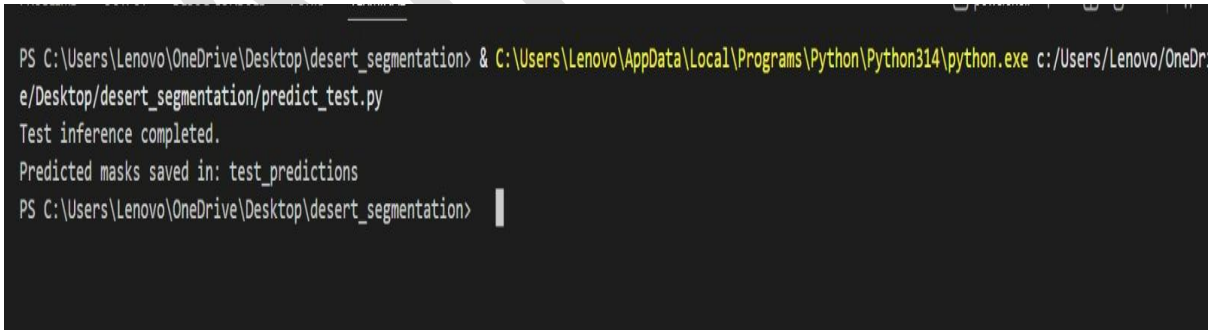
$$IoU = \frac{Intersection}{Union}$$

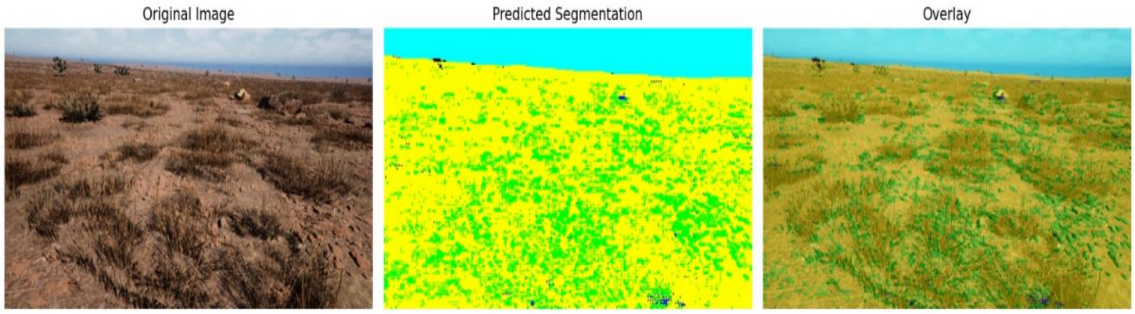
This metric quantifies pixel-level prediction fidelity and remains the primary judging criterion in the hackathon.

The evaluation process:

1. Generate predictions via argmax over class logits.
2. Compute per-class intersection and union.
3. Aggregate IoU across dataset batches.

IoU provides a more reliable indicator than accuracy, especially in the presence of class imbalance.





7. Strengths of the Implementation

- Clean modular architecture
- Explicit preprocessing pipeline
- Deterministic class mapping
- Reproducible training structure
- Dedicated evaluation utilities
- Lightweight and hardware-efficient design

The separation of dataset, model, training, and utilities reflects sound software engineering principles.

8. Identified Limitations

While stable and reproducible, the current baseline exhibits limitations:

- Shallow network depth
- Absence of skip connections
- No advanced regularization
- No data augmentation
- Limited multi-scale feature extraction

These constraints may impact segmentation boundary precision and generalization to highly varied terrain textures.

9. Future Improvements

To enhance performance:

1. Introduce data augmentation for improved domain generalization.
2. Integrate skip connections (U-Net style).
3. Employ Batch Normalization layers.
4. Explore Dice or Focal Loss for class imbalance.
5. Increase network depth for richer feature representation.
6. Experiment with learning rate scheduling.

10. Conclusion

This project delivers a structured and reproducible baseline segmentation system for desert terrain understanding. By emphasizing modularity, evaluation transparency, and architectural clarity, we established a strong foundation for iterative experimentation.

The system demonstrates the full semantic segmentation pipeline:

Data Engineering → Model Construction → Training → IoU Evaluation

Although minimalistic by design, this framework provides a stable launchpad for performance optimization and architectural innovation.

Through disciplined engineering and metric-driven evaluation, this project contributes meaningfully to offroad autonomy research within synthetic desert environments.